

本科毕业设计（论文）

LabGuardian 智能电路实验助教系统

An Intelligent Circuit Experiment Tutor System Based on Edge AI

项目名称: LabGuardian 智能电路实验助教系统

文档类型: 毕业论文式项目报告初稿

代码仓库: LabGuardian-Server

文档仓库: LabGurdian-latex

学生姓名:

指导教师:

所在学院:

提交日期: 2026 年 5 月

摘要

LabGuardian 是一套面向高校电路与模拟电子实验教学场景的智能实验助教系统。系统以“图像采集—元件检测—引脚定位—孔位映射—拓扑重构—参考比较—智能诊断—教师课堂管理”为主线，将边缘视觉识别、图结构比较、知识检索和诊断 Agent 编排统一到一个后端服务中。当前后端采用 Python 3.11 与 FastAPI 构建统一 API，使用 Pydantic 定义请求与响应契约，使用 Celery 与 Redis 支持异步任务，使用 YOLO 系列模型进行元件与引脚检测，使用 NetworkX 表示参考电路与当前电路的逻辑图，并在课堂状态、硬件遥测、知识库检索和诊断问答之间建立了工程化的数据闭环。

本文按照毕业论文式结构，对 LabGuardian 的研究背景、需求分析、总体架构、关键模块设计、核心算法流程、接口设计、部署方案、测试与改进方向进行系统阐述。报告初稿重点依据当前 LabGuardian-Server 代码仓库中的后端实现撰写：入口文件 `app/main.py` 统一注册课堂、流水线、参考电路、知识库、拓扑、Agent 与 WebSocket 模块；`app/pipeline/orchestrator.py` 负责串联 S1 检测、S1.5 引脚检测、S2 映射、S3 拓扑、S4 验证和 S5 语义分析；`docs/comparison-architecture.md` 描述以 Python DSL 参考电路、逻辑图构建、拓扑归一化和图同构比较为核心的验证架构；`docs/telemetry-protocol.md` 规定了 DK-2500 边缘端 CPU、内存、iGPU、NPU 与流水线阶段的实时遥测协议；`docs/retrieval-contract.md` 定义了教学场景、故障案例、数据手册与电路知识库的合法检索来源和训练部署一致性约束。

本文认为，LabGuardian 的主要工程价值在于将传统电路实验中“教师巡查式纠错”转化为“视觉感知驱动的实时拓扑诊断”。系统不只识别元件类别，还把面包板孔位、电气网络、参考电路拓扑、错误类型和教学解释连接起来，使学生能够在实验过程中获得及时反馈，教师能够在课堂中获得全班风险、进度和指导记录。后续工作需要继续完善真实硬件环境下的稳定性评测、数据集规模、端侧模型加速、论文实验数据和前后端联调截图。

关键词： 智能实验助教；边缘人工智能；电路拓扑识别；面包板视觉检测；图同构比较；教学诊断 Agent

Abstract

LabGuardian is an intelligent tutoring system for hands-on circuit experiments. It integrates edge visual perception, breadboard pin mapping, topology reconstruction, graph-based validation, knowledge-grounded diagnosis, classroom monitoring and hardware telemetry into a unified backend service. The current server is implemented with Python 3.11 and FastAPI. Pydantic is used for typed API contracts, Celery and Redis are used for asynchronous pipeline jobs, YOLO-based models are used for component and pin detection, and NetworkX-based logical graphs are used to compare the observed circuit against a reference circuit.

This report follows a graduation-thesis style and presents the background, requirements, architecture, module design, algorithmic workflow, API design, deployment plan, testing strategy and future improvements of LabGuardian. The draft is based on the current backend repository. The system entry point registers the classroom, pipeline, reference, knowledge-base, topology, Agent and WebSocket routers. The pipeline orchestrator executes detection, pin detection, mapping, topology reconstruction, validation and semantic analysis. The comparison architecture uses Python DSL reference circuits, logical graph

construction, net normalization and topology-aware graph comparison. The telemetry protocol streams edge-device resource usage and current pipeline stage to WebSocket subscribers. The retrieval contract further constrains the legal knowledge sources for teaching scenes, fault cases, datasheets and circuit knowledge, ensuring consistency between training and deployment.

The main contribution of LabGuardian is to transform manual classroom inspection into real-time topology diagnosis driven by visual perception and structured reasoning. The system connects component recognition, breadboard coordinates, electrical nets, reference topology, error categories and pedagogical explanations, enabling immediate feedback for students and dashboard-level supervision for teachers.

Keywords: intelligent tutoring system; edge AI; circuit topology recognition; breadboard vision; graph comparison; diagnostic agent

目录

1	绪论	6
1.1	研究背景与行业痛点	6
1.2	研究意义	6
1.3	国内外相关技术概述	7
2	需求分析	7
2.1	用户角色与使用场景	7
2.2	功能性需求	8
2.3	非功能性需求	8
3	总体架构设计	9
3.1	系统分层架构	9
3.2	统一后端入口	9
3.3	技术栈选型	9
4	核心模块设计与实现	10
4.1	Pipeline 流水线设计	10
4.2	面包板孔位与板卡 Schema	11
4.3	参考电路表示与图比较	11
4.4	最小人工标注与手动修正	11
4.5	课堂状态与教师端功能	11
4.6	诊断 Agent 与知识检索约束	12
4.7	硬件遥测与边缘部署支持	12
5	接口与数据模型设计	12
5.1	Pipeline API	12
5.2	Classroom API	13
5.3	Agent API	13
5.4	遥测与 WebSocket API	13
6	关键算法与实现流程	14
6.1	元件检测与引脚检测	14
6.2	孔位吸附与拓扑重构	14
6.3	图同构与错误报告	14
6.4	语义分析与教学解释	14
7	部署与运行方案	15
7.1	本地开发运行	15
7.2	Docker Compose 部署	15
7.3	边缘设备部署	15
8	测试方案与结果评价	15
8.1	单元测试与接口测试	15
8.2	端到端评价指标	16

8.3 当前不足	16
9 总结与展望	16
10 参考文献	16
11 附录 A：代码依据清单	17
12 附录 B：后续补充材料清单	17

1 绪论

1.1 研究背景与行业痛点

高校电子类基础实验包括电路分析、模拟电子技术和数字电子技术等课程，是培养学生工程素养和实践能力的核心环节。学生通常需要在面包板上搭建由电阻、电容、二极管、三极管、运放、计时器等器件构成的基础电路，并通过示波器或万用表观察电压、电流和波形。然而在真实课堂中，实验教学不仅面临 1:30 以上师生比带来的指导压力，也面临物理实操层面的结构性困难：同一个电路故障可能来自元件缺失、引脚插错、极性接反、电源轨误接、跨孔位短接、信号节点悬空或参考电路选择错误。教师在有限时间内需要同时巡查多个工位，很难持续跟踪每个学生的搭建过程与错误演化。

第一，空间遮挡会直接限制电路拓扑提取。真实电路实验中，错综复杂的杜邦线常常遮挡底层元器件与引脚位置，传统纯视觉目标检测算法虽然可以识别电阻、芯片等较大目标，但面对引脚这类尺度小、遮挡强、语义依赖上下文的目标时，容易出现特征丢失、定位偏移和连接关系误判，难以还原真实电气网络。第二，电路逻辑诊断与微观缺陷研判存在门槛。初学者需要把二维原理图映射到三维面包板空间，稍有极性反接、短路或电源轨误接，就可能造成芯片损坏；部分教学场景中相关损坏率可达 8%–15%，说明仅靠事后测量和人工巡查难以及时控制风险。第三，当前教学工具多停留在面包板搭建与仿真验证阶段，对后续 PCB 原型设计、焊接制造和检验环节支持不足，难以形成覆盖“原型设计—搭建验证—制造检验”的电子工程教学闭环。

随着边缘人工智能和轻量化视觉模型的发展，实验现场具备了将摄像头、嵌入式算力、规则化电路知识和教学反馈结合起来的条件。LabGuardian 项目正是面向这一需求提出的智能电路实验助教系统。它不是单纯的目标检测系统，也不是单纯的大模型问答系统，而是一个以电气拓扑为核心中间表示的工程系统：视觉模块负责从图像中抽取元件、引脚和孔位；拓扑模块负责把孔位映射为可比较的电气网络；验证模块负责把当前电路与参考电路进行结构化比较；Agent 模块负责把错误证据、知识库和安全提示转化为可理解的教学反馈。

因此，LabGuardian 将学习反馈从“事后纠错”前移至“过程引导”，并进一步面向“面包板原型验证—PCB 成品制造”的全周期教学辅助闭环展开设计。系统借助 PCM (Push-Based Context Management, 推送式上下文管理) 技能挂载机制和 RAG 芯片知识库，使芯片型号与实验场景能够灵活扩展；新增元器件时，可优先通过导入技术手册补充本地知识，再结合视觉与拓扑模块适配新的诊断任务。全部推理与知识检索在 DK-2500 本地离线完成，可满足实验室弱网环境、数据隐私和现场稳定性要求。项目的主要演示链路可概括为“多模态感知 + 知识增强异构推理 + 教师协同看板”，评审或教师在不额外依赖外网的情况下即可观察从图像输入、拓扑诊断到课堂看板反馈的端到端效果。

1.2 研究意义

LabGuardian 的研究意义主要体现在三个方面。

第一，在教学层面，系统可以降低基础实验的反馈延迟。传统实验中，学生往往只有在测量结果异常或教师巡查时才发现错误。系统若能在搭建过程中实时指出“某个元件引脚落在错误孔位”

“某个输出节点未与参考电路一致”“某个极性元件疑似接反”，就能帮助学生把错误定位到具体物理位置和电气原因。

第二，在工程层面，系统把计算机视觉、图算法、Web 服务、异步任务、知识检索和课堂管理整合到统一架构中。项目代码显示，后端已经具备统一 API 入口、模型路径配置、流水线编排、图比较、课堂状态管理、Agent 问答、硬件遥测和 Docker Compose 部署基础。这种模块化结构便于后续替换模型、扩展电路类型、增加前端交互和迁移到边缘设备。

第三，在研究层面，系统提供了一个从“非结构化图像”到“结构化电路语义”的完整链路。与只输出检测框的视觉系统不同，LabGuardian 进一步构建 netlist_v2、逻辑参考图和比较报告，使得系统输出可以被验证、追踪和解释。这为毕业设计中的算法分析、实验评测和消融研究提供了明确对象。

1.3 国内外相关技术概述

从技术路线看，本项目涉及四类相关工作。第一类是电子实验辅助系统，主要关注实验步骤展示、仿真验证和在线题库，但通常缺少对真实面包板搭建状态的实时感知。第二类是目标检测和关键点检测技术，包括 YOLO、OpenCV 和图像几何校准方法，可用于定位元件和引脚。第三类是电路拓扑分析与网表比较技术，可将物理连接关系抽象为节点和边，并通过图匹配、同构和编辑距离判断结构差异。第四类是知识增强诊断系统，包括 RAG、规则引擎、ReAct Agent 和反思校验节点，用于把结构化错误转化为自然语言解释。

LabGuardian 的特点在于组合这些技术，并把“当前电路是否正确”落实到工程可执行的 API 和数据结构中。系统既保留确定性模块，例如孔位映射、图构建、图比较和规则校验；也为可选的智能模块预留接口，例如数据手册检索、本地 VLM、OpenVINO 端侧模型和诊断 Agent。总体来看，当前国内外仍缺少专门面向面包板电路实验场景、同时融合计算机视觉、图论分析与视觉大语言模型的开源 AI 辅助教学系统，这也是 LabGuardian 的主要创新切入点。

2 需求分析

2.1 用户角色与使用场景

系统面向三类主要用户。第一类是学生端用户。学生在面包板上搭建实验电路，上传 1 至 3 张俯拍图像，系统返回元件数量、引脚孔位、网络连接、风险等级、诊断文本和可视化所需数据。第二类是教师端用户。教师需要查看全班工位状态、进度排行、风险警报、缩略图和指导审计记录，并向单个工位或全班推送指导信息。第三类是系统维护者。维护者需要配置模型路径、知识库路径、Redis/Celery、端侧设备、遥测采样和参考电路资源。

角色	核心诉求	系统响应
学生	快速确认电路搭建是否正确	返回检测结果、拓扑网表、风险等级和诊断建议
教师	掌握全班实验进度与风险	提供工位状态、排行、警报、缩略图和指导推送
维护者	保证模型、知识库和服务稳定运行	提供配置项、部署文件、健康检查和遥测协议

表 1 系统角色与需求

2.2 功能性需求

结合后端代码，LabGuardian 的核心功能性需求可概括如下。

1. 图像输入与检测：系统应接受 1 至 3 张面包板俯拍图片，支持置信度、NMS IoU、推理尺寸等检测参数，并输出元件检测结果。
2. 引脚定位与孔位映射：系统应对元件引脚进行定位，并通过面包板校准与网格吸附，把图像坐标转换为 `hole_id`、`logic_loc` 和 `electrical_node_id`。
3. 拓扑重构：系统应基于面包板导通规则、电源轨配置和元件引脚连接，生成当前电路的 `netlist_v2` 与可视化拓扑图。
4. 参考电路比较：系统应支持通过 `reference_id` 加载参考电路，或接受内联参考电路 `payload`，并将当前拓扑与参考逻辑图进行比较，输出相似度、进度、风险等级和诊断条目。
5. 手动修正与重算：当视觉或孔位映射出现误差时，前端应能提交手动孔位修正、端口标注、网络别名、网络合并和三极管极性选择，后端重跑拓扑和验证阶段。
6. 课堂状态管理：系统应接收学生端心跳，维护工位状态、缩略图、进度、风险和参考电路，并支持教师查询和广播指导。
7. 智能诊断问答：系统应根据课堂状态、当前错误、知识库和白盒工具，生成面向学生的诊断反馈，并提供任务提交与状态查询 API。
8. 硬件遥测：系统应在边缘设备上周期性采样 CPU、内存、iGPU、NPU 和当前 `pipeline stage`，并通过 WebSocket 推送给前端仪表盘。

2.3 非功能性需求

在非功能性需求方面，系统需要满足以下要求。

可靠性方面，视觉模型、拓扑推断、知识检索或硬件采样出现局部异常时，不应导致整个服务崩溃。可选模块应具备降级策略，例如遥测字段缺失时返回 `null`，GNN 拓扑分类低置信时不写入错误场景，Agent 在无场景时跳过对应知识检索。

实时性方面，课堂实验需要接近实时的反馈。同步接口用于演示和快速验证，异步接口通过 Celery/Redis 支持较重的视觉任务。硬件遥测默认以 5 Hz 推送，可覆盖课堂仪表盘需求。

可解释性方面，系统输出不应只是“正确/错误”，还应包括元件、引脚、孔位、网络、错误码、证据引用和安全提示，使学生能够据此修改电路。

可部署性方面，后端应能在开发机和边缘设备之间迁移。`docker-compose.yml` 已将 `Redis`、`server` 和 `worker` 拆分为独立服务，并通过只读挂载 `models` 目录配置元件检测和引脚检测模型。

3 总体架构设计

3.1 系统分层架构

LabGuardian 后端采用分层设计。最外层为 FastAPI 路由层，提供课堂、流水线、参考电路、拓扑、知识库、Agent、WebSocket 和遥测接口。应用服务层封装 `PipelineService`、`AgentService`、`ClassroomState`、`GuidanceService`、`ReferenceService` 等服务对象。领域层负责板卡 `schema`、逻辑参考电路、网表归一化、图比较、DSL 编译和拓扑标签。流水线层负责视觉检测、引脚检测、孔位映射、拓扑重构、验证和语义分析。基础设施层提供配置、Celery、Redis、模型路径、静态知识资源和 Docker 部署。

层次	代表模块	主要职责
接口层	<code>app/api/v1/*.py</code>	对外提供 REST 与 WebSocket 能力
应用服务层	<code>app/services/*.py</code>	编排流水线、课堂状态、Agent 与指导推送
领域层	<code>app/domain/*</code>	表达参考电路、板卡、电气网络和图比较规则
流水线层	<code>app/pipeline/*</code>	完成视觉检测、映射、拓扑、验证和语义分析
基础设施层	<code>app/core/*</code> , <code>docker-compose.yml</code>	配置、异步任务、部署和运行环境支持

表 2 LabGuardian 后端分层架构

3.2 统一后端入口

系统入口位于 `app/main.py`。该文件创建 FastAPI 应用，设置项目名称、描述、版本、OpenAPI 路径和文档路径；通过应用生命周期函数启动与停止遥测服务；添加 CORS 中间件；挂载 `/knowledge` 静态资源；并注册 `classroom`、`pipeline`、`references`、`angnt`、`kb`、`topology`、`websocket` 和 `telemetry_ws` 等路由模块。该入口还提供 `/health` 健康检查和 `/version` 版本信息接口。

这种设计使后端从一开始就具备“统一服务”的形态：前端只需要连接一个服务器即可访问电路分析、课堂管理、诊断问答、知识资源和硬件遥测。对于毕业设计实现而言，统一入口还方便进行端到端演示和部署验收。

3.3 技术栈选型

项目配置文件 `pyproject.toml` 体现了系统的主要技术栈：Web 框架使用 FastAPI、Uvicorn、Pydantic 和 WebSocket；异步任务使用 Celery 与 Redis；视觉与图处理使用 Ultralytics、OpenCV、NumPy、NetworkX 和 scikit-image；数据存储预留 SQLAlchemy 与 Alembic；知识库方向引入 LangChain、Chroma、pypdf 与 tiktoken；边缘部署方向预留 OpenVINO、OpenVINO GenAI 与本地嵌入模型。

类别	主要依赖	用途
Web 服务	FastAPI, Uvicorn, Pydantic	API、数据校验与服务运行
异步任务	Celery, Redis	长耗时 pipeline 任务提交与状态查询
视觉识别	Ultralytics, OpenCV, NumPy	元件检测、引脚定位与图像处理
图算法	NetworkX, scikit-image	拓扑图构建、图比较与几何辅助
知识检索	LangChain, Chroma, pypdf	数据手册、教学场景和故障案例知识库
边缘 AI	OpenVINO, tokenizers	端侧嵌入、VLM 与 LLM 推理预留

表 3 后端技术栈概览

4 核心模块设计与实现

4.1 Pipeline 流水线设计

系统最核心的模块是 `app/pipeline/orchestrator.py` 中的 `run_pipeline`。该函数串联 S1、S1.5、S2、S3、S4 和 S5 六个阶段，并通过 `PipelineContext` 复用元件检测器与引脚检测器，避免每次请求重复加载模型。各阶段职责如下。

阶段	名称	职责
S1	元件检测	调用 <code>ComponentDetector</code> 对输入图像进行元件识别，输出检测框和类别信息
S1.5	引脚检测	调用 <code>PinRoiDetector</code> 或整图 <code>pin</code> 模型识别元件引脚
S2	孔位映射	通过面包板校准器和 <code>Snap-to-Grid</code> 策略把引脚映射到孔位
S3	拓扑重构	根据元件引脚、板卡导通规则和电源轨配置生成拓扑图与 <code>netlist_v2</code>
S4	参考验证	将当前拓扑与参考电路比较，生成相似度、进度和风险报告
S5	语义分析	根据网表和参考电路推断电路类型、错误语义和教学解释线索

表 4 流水线阶段划分

从数据流角度看，系统把图片逐步转化为结构化事实：图像首先变为元件检测结果；检测结果与引脚模型结合形成元件—引脚集合；引脚坐标经校准后映射为面包板孔位；孔位根据板卡 `schema` 转化为电气节点；电气节点聚合为网络；网络与元件连接形成当前网表；最后当前网表与参考电路图进行比较，得到可解释诊断结果。

```
images_b64
-> S1 detections
-> S1.5 components with pins
-> S2 mapped hole_id / logic_loc / electrical_node_id
-> S3 netlist_v2 + topology_graph
```

```
-> S4 comparison_report + risk_level
-> S5 semantic_analysis
```

4.2 面包板孔位与板卡 Schema

面包板场景的难点在于，视觉坐标并不能直接表示电气连接。项目通过 BoardSchema 把 hole_id 映射为稳定的 electrical_node_id。默认 schema 覆盖 63 行双侧主区和电源轨分段，并兼容历史命名。系统配置中 BREADBOARD_ROWS 默认为 63，BREADBOARD_COLS_PER_SIDE 默认为 5，说明当前主要面向 63 行、左右各 5 列的比赛/教学面包板。

板卡 schema 的核心作用是把物理孔位抽象为导通节点。例如主区 A1-E63 和 F1-J63 根据行号和左右侧归入不同电气节点；电源轨按 LP, LN, RP, RN 及其分段规则归入电源网络。这样，视觉阶段只需尽量准确给出孔位，后续拓扑阶段即可依据确定性规则恢复电气连接。

4.3 参考电路表示与图比较

项目的参考电路不再要求用户手写完整 JSON，而是支持使用 Python DSL 描述参考电路。参考电路源文件位于 knowledge/references/*.py，通过 Circuit, Net, Component, Pin 等对象描述电路，再编译为 logical_reference_v1 payload。系统把参考电路和当前 netlist_v2 都转化为 NetworkX 无向二分图，其中 component node 表示元件，net node 表示电气网络，edge 表示元件引脚与网络的连接关系。

图比较模块的关键思想是“拓扑优先，标签辅助”。电源和地需要严格匹配；输入输出端口在参考侧存在关键 label 时严格检查；内部 signal 不依赖人工 label，而是通过拓扑结构推断；电阻、陶瓷电容和导线等无极性两脚器件忽略引脚顺序；三极管、电位器、LED、二极管和电解电容等极性或功能引脚器件保持严格匹配。比较流程会先尝试完整图同构，失败后进行角色推断、包含/子图判断，最后用图编辑距离或 fallback 生成错误报告。

4.4 最小人工标注与手动修正

为降低课堂使用负担，系统设计了“最小人工标注”协议。前端通常只需要让用户指定电源轨和输入/输出端口，而不要求用户为内部信号节点命名。PortAnnotation 仅包含 role、target、label 和 source，其中 target 可通过孔位、元件引脚、电气网络或图像坐标定位。

当视觉检测或孔位映射出现偏差时，系统提供 /api/v1/pipeline/recompute-corrected 接口，允许前端提交 ManualCorrectionPatch、ManualNetRoleAssignment、ManualNetAliasAssignment、ManualNetMergeAssignment 和 ManualPinPolarityAssignment。后端应用修正后重跑 S3/S4，并把修正信息写入 runtime_metadata，保证结果可追溯。

4.5 课堂状态与教师端功能

课堂模块位于 app/api/v1/classroom.py。学生端通过 POST /api/v1/classroom/heartbeat 每隔约 2 秒上报工位状态，后端维护每个 station 的缩略图、元件数量、网络数量、进度、风险等级、诊断结果和拓扑标签。教师端可通过 /stations、/ranking、/alerts、/stats 和 /station/{station_id} 获取全班或单工位信息。

教师指导模块还支持向单个学生发送指导消息，或向全班广播消息，并通过 `/guidance/audit` 查询指导记录。该设计使系统不仅能自动诊断，也能成为教师课堂管理工具：自动模块负责发现风险，教师模块负责人工介入和教学组织。

4.6 诊断 Agent 与知识检索约束

Agent 模块通过 `POST /api/v1/angnt/ask` 提交问答任务，通过 `GET /api/v1/angnt/status/{job_id}` 查询状态。根据 `docs/pcm-agent-architecture.md`，当前 Agent 架构强调 **Push-Based Context Management**：先从 `ClassroomState` 抽取运行时证据，再根据错误码和错误标签构建 `ContextPack`，随后只允许调用白名单中的 **deterministic tools**，最后生成草稿答案并进行规则化校验和修复。

知识检索契约进一步规定，生产 Agent 和蒸馏管线只允许使用 `teaching_scene`、`fault_case`、`datasheet_v2` 和 `circuit_kb` 等合法来源；旧 PDF/Chroma 知识库和云向量空间仅可作为 `admin/debug` 工具，不得进入主链路。该约束可以减少错误场景污染和训练部署不一致，有利于保证诊断答案的可靠性与可复现性。

4.7 硬件遥测与边缘部署支持

硬件遥测协议用于在 DK-2500 等边缘设备上采样 CPU、内存、iGPU、NPU 和当前 pipeline stage。系统通过 `/ws/telemetry/system` 以 WebSocket 推送实时帧，通过 `/api/v1/telemetry/latest` 提供最新帧的 REST 查询。遥测帧版本为 `telemetry_frame_v1`，默认采样频率由 `TELEMETRY_HZ` 控制，当前配置默认 5.0 Hz。

遥测模块的设计具有典型的工程降级特征：硬件不存在或采样失败时对应字段返回 `null`，客户端慢时丢弃旧帧而不阻塞采样器，长时间无帧时发送 `heartbeat`。这些策略适合课堂演示环境，因为它们优先保证主业务 pipeline 不被遥测模块拖垮。

5 接口与数据模型设计

5.1 Pipeline API

流水线 API 位于 `app/api/v1/pipeline.py`，主要接口如下。

接口	方法	说明
/api/v1/pipeline/run	POST	同步执行完整 pipeline，适合演示和调试
/api/v1/pipeline/submit	POST	提交异步 pipeline 任务，需要 Celery 与 Redis
/api/v1/pipeline/status/ {job_id}	GET	查询异步任务状态与结果
/api/v1/pipeline/analyze	POST	返回元件引脚二维定位、网表和拓扑图
/api/v1/pipeline/compare- netlist	POST	直接比较参考电路与 current_netlist_v2，用于 调试
/api/v1/pipeline/recompute- corrected	POST	应用前端手动修正并重跑拓扑与验证
/api/v1/pipeline/visualize/ ports	POST	返回端口映射和可视化表格所需数据

表 5 Pipeline 主要接口

PipelineRequest 是 pipeline 的核心请求模型，包含 station_id、images_b64、conf、iou、imgsz、reference_id、reference_circuit、rail_assignments、port_annotations、net_role_assignments、net_alias_assignments 和 net_merge_assignments。该结构既覆盖自动识别，也覆盖用户标注和调试场景。

PipelineResult 是统一响应模型，包含 job 信息、阶段结果、总耗时、元件数、网络数、进度、相似度、诊断列表、比较报告、风险等级、风险原因和运行时元数据。该模型把各阶段结果统一包装，便于前端在一个对象中获取全链路信息。

5.2 Classroom API

课堂 API 支持学生端心跳、教师端统计、指导推送和参考电路设置。其设计重点是“状态聚合”而不是“复杂查询”。学生持续上报状态后，教师端无需直接访问 pipeline 原始任务即可查看全班概况。课堂状态还为 Agent 提供上下文来源，使诊断问答能够知道当前工位、当前错误、当前参考电路和最近一次图像缩略图。

5.3 Agent API

Agent API 当前保持最小骨架：提交问题和查询状态。这样的接口设计有利于后续扩展不同回答模式，例如概念讲解、实验指导、诊断解释和混合问答。Agent 内部可迭代升级为 LangGraph/ReAct、本地 LLM 或 OpenVINO GenAI，而前端接口不需要频繁变化。

5.4 遥测与 WebSocket API

系统包括课堂 WebSocket 和遥测 WebSocket。遥测主通道 /ws/telemetry/system 用于向仪表盘推送硬件占用和流水线阶段。课堂 WebSocket 则可用于前端订阅工位变化、指导消息或实时结果。WebSocket 接口减少轮询开销，也更符合课堂实时大屏和实验过程追踪需求。

6 关键算法与实现流程

6.1 元件检测与引脚检测

系统检测层由 `ComponentDetector` 和 `PinRoiDetector` 提供模型能力。配置中 `YOLO_MODEL_PATH` 指向元件检测模型, `PIN_MODEL_PATH` 指向引脚检测模型; `YOLO_CONF_THRESHOLD`、`YOLO_IOU_THRESHOLD` 和 `YOLO_IMGSZ` 分别控制置信度、NMS 阈值和输入尺寸。默认设备为 CPU, 后续可以根据边缘硬件情况切换到 GPU 或 OpenVINO 推理路径。

该阶段输出的关键不是最终答案, 而是高质量中间事实: 元件类别、检测框、引脚坐标、候选孔位和置信度。系统后续模块可根据这些事实进行几何校准、投票、人工修正和拓扑推断。

6.2 孔位吸附与拓扑重构

S2 映射阶段的目标是把引脚从图像坐标吸附到面包板网格。系统配置中提供 `PIN_CANDIDATE_K`、`MAPPING_DIST_GATE_LOWER`、`MAPPING_DIST_GATE_UPPER`、`MAPPING_VOTE_RANK_DECAY` 和 `MAPPING_AMBIGUOUS_MARGIN` 等参数, 用于控制候选孔范围、多视图投票衰减和歧义判定。此设计体现出项目对真实图像误差的考虑: 系统不是简单取最近孔位, 而是保留候选、投票和歧义信息。

S3 拓扑阶段根据元件引脚孔位、板卡导通规则和电源轨分配构建 `netlist_v2`。默认电源轨配置为 `top_plus = VCC`、`top_minus = VEE`、`bot_plus = GND`、`bot_minus = GND`, 学生端可通过请求覆盖。拓扑阶段输出不仅用于验证, 也用于前端可视化和 Agent 诊断。

6.3 图同构与错误报告

S4 验证阶段将当前拓扑与参考逻辑图比较。若当前图与参考图完全同构, 则可判定为逻辑正确; 若包含参考图但存在额外连接或元件, 可判定为接近正确但有冗余风险; 若是参考图子图, 则可判定为未完成; 若明显不匹配, 则生成错接报告。报告中包含 `logic_correct`、`similarity`、`progress`、`diagnostics` 和结构化 `comparison_report`。

相比基于字符串或节点名称的比较, 图比较的优势是能容忍内部信号名称变化和无极性器件顺序交换。对于教学场景而言, 学生不关心某个内部网络被叫作 `NET_001` 还是 `VLP`, 真正关键的是元件之间是否形成了正确的电气连接。

6.4 语义分析与教学解释

S5 语义分析把拓扑结果进一步转化为面向教学的解释线索。例如系统可以根据网表和参考电路推断电路类型、错误家族、缺失元件、极性问题或保护电阻缺失等信息。语义分析结果会同步到课堂状态, 并可作为 Agent 检索教学场景、故障案例和数据手册的条件。

7 部署与运行方案

7.1 本地开发运行

本地开发时，可安装 Python 3.11 环境并执行项目依赖安装。服务入口为 `uvicorn app.main:app --port 8000`。若只使用同步接口 `/api/v1/pipeline/run`，可以先不启动 Celery worker；若使用异步任务接口，则需要 Redis 和 worker。

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
celery -A app.core.celery_app:celery_app worker -Q pipeline -c 1 --loglevel=info
```

7.2 Docker Compose 部署

项目提供 `docker-compose.yml`，定义了 `redis`、`server` 和 `worker` 三个服务。`server` 暴露 8000 端口，`worker` 使用 Celery 消费 `pipeline` 队列，二者都通过环境变量指定模型目录，并把宿主机 `./models` 以只读方式挂载到容器 `/app/models`。这种部署方式适合课堂演示：`Redis` 负责任务队列，`server` 对接前端，`worker` 专门运行视觉 `pipeline`。

7.3 边缘设备部署

在 DK-2500 等边缘设备部署时，重点需要配置模型路径、推理设备、OpenVINO 模型目录和遥测开关。系统已经为元件检测、引脚检测、数据手册嵌入模型、VLM 和 Agent LLM 预留了配置项。实际部署时应优先保证基础视觉 `pipeline` 稳定运行，再逐步启用本地嵌入、VLM 微观解释和端侧 LLM。

8 测试方案与结果评价

8.1 单元测试与接口测试

由于本报告为初稿，具体测试数量和覆盖率应以后续 CI 输出为准。建议将测试分为以下四类：第一，领域模型测试，覆盖 `BoardSchema`、`net normalization`、`reference DSL` 编译和 `graph compare`；第二，流水线阶段测试，覆盖 `S2` 映射、`S3` 拓扑、`S4` 验证和手动修正重算；第三，API 测试，覆盖 `classroom`、`pipeline`、`angnt`、`telemetry` 等接口的正常与异常请求；第四，知识检索与 Agent 测试，覆盖 `scene_resolver`、`fault_case`、`datasheet_v2`、`verification` 和 `retrieval contract`。

测试类别	测试对象	评价指标
视觉与映射	检测、引脚、孔位吸附	孔位准确率、歧义率、人工修正后通过率
拓扑与验证	netlist_v2、图同构、错误报告	正确判定率、错误定位准确率、误报/漏报率
API 与课堂	REST、WebSocket、课堂状态	响应时间、并发工位数、状态一致性
Agent 与检索	ContextPack、工具白名单、知识源	引用正确率、wrong-scene rate、fallback rate
边缘部署	Docker、遥测、模型推理	端到端延迟、资源占用、降级稳定性

表 6 建议测试维度

8.2 端到端评价指标

毕业论文最终版建议补充真实实验数据，包括：不同电路类型下的端到端正确率；不同拍摄角度、光照和遮挡条件下的孔位映射准确率；错误类型级别的召回率；同步接口和异步接口的平均耗时；DK-2500 上 CPU、内存、iGPU、NPU 占用曲线；教师端大屏在多工位并发下的刷新延迟；学生根据诊断建议完成修正的平均次数。

8.3 当前不足

当前后端代码已经形成较完整的工程骨架，但仍有若干需要在论文最终版中补齐的内容。第一，视觉模型的数据集来源、训练过程、验证集指标和混淆矩阵需要明确。第二，端侧部署实验尚需给出 DK-2500 实测延迟和资源占用。第三，系统仍需更多真实课堂样本验证复杂接线、遮挡和错误组合场景。第四，前后端交互截图、用户流程和教师端 dashboard 需要补充。第五，毕业论文需要将工程描述进一步转化为可量化实验和对比分析。

9 总结与展望

本文基于当前 LabGuardian 后端代码，按照毕业论文式结构整理了项目背景、需求、架构、模块、接口、算法、部署和测试方案。系统以 FastAPI 为统一后端入口，以六阶段 pipeline 为核心数据处理链路，以 BoardSchema 和 netlist_v2 为电气拓扑中间表示，以图比较作为逻辑验证方法，以课堂状态和诊断 Agent 作为教学反馈接口，并通过硬件遥测和 Docker Compose 支持边缘部署。

总体来看，LabGuardian 的创新点不在单个模型或单个接口，而在于把实验场景中的多个异构问题统一到“结构化拓扑诊断”这一主线中。视觉模块负责看见真实电路，拓扑模块负责理解电气连接，比较模块负责判断是否符合参考电路，Agent 模块负责把错误转化为教学语言，课堂模块负责支撑教师管理。这种架构具有良好的可扩展性：未来可增加更多电路模板、更多元件类别、更强的端侧模型、更精细的故障案例库和更完善的前端可视化。

后续工作建议包括：一是完善数据集和实验评测，形成可复现的准确率、召回率和延迟数据；二是继续优化端侧推理，包括 OpenVINO、NPU、INT8/INT4 量化和模型缓存；三是完善前端交互，使手动修正、端口标注、网络合并和诊断解释更自然；四是扩大知识库覆盖范围，并严格保持检索契约，避免 wrong-scene 和 legacy fallback；五是將课堂实际试用反馈纳入迭代，验证系统对学生学习效率 and 教师巡查负担的影响。

10 参考文献

- [1] FastAPI Documentation. FastAPI Web Framework.
- [2] Ultralytics YOLO Documentation. Object Detection and Pose Estimation.
- [3] NetworkX Documentation. Graph Algorithms and Data Structures.
- [4] OpenVINO Documentation. Edge AI Inference and Model Optimization.

[5] LabGuardian-Server 代码仓库: `app/main.py`, `app/pipeline/orchestrator.py`, `app/api/v1/pipeline.py`, `app/schemas/pipeline.py`, `docs/comparison-architecture.md`, `docs/telemetry-protocol.md`, `docs/retrieval-contract.md`.

11 附录 A：代码依据清单

文件	本报告使用的信息
<code>pyproject.toml</code>	项目名称、Python 版本、依赖、可选 <code>edge/embedding/gnn</code> 组件
<code>app/main.py</code>	FastAPI 入口、路由注册、CORS、静态知识目录、健康检查与版本接口
<code>app/core/config.py</code>	模型路径、检测参数、板卡参数、知识库、遥测和 Agent 配置
<code>app/pipeline/orchestrator.py</code>	S1 至 S5 流水线编排、共享模型上下文、运行时元数据
<code>app/services/pipeline_service.py</code>	同步/异步 <code>pipeline</code> 、课堂状态同步、手动修正重算
<code>app/api/v1/pipeline.py</code>	<code>pipeline</code> REST 接口、比较接口和可视化端口接口
<code>app/schemas/pipeline.py</code>	<code>PipelineRequest</code> 、 <code>PipelineResult</code> 、 <code>PortAnnotation</code> 和手动修正数据模型
<code>docs/comparison-architecture.md</code>	参考 DSL、逻辑图构建、 <code>net</code> 归一化和图比较策略
<code>docs/board-schema-format.md</code>	<code>BoardSchema</code> 、 <code>hole_id</code> 到 <code>electrical_node_id</code> 映射和最小标注协议
<code>docs/pcm-agent-architecture.md</code>	诊断 Agent、 <code>ContextPack</code> 、工具白名单和 <code>ReAct/Reflection</code> 流程
<code>docs/retrieval-contract.md</code>	合法检索源、训练部署一致性和蒸馏前置约束
<code>docs/telemetry-protocol.md</code>	边缘硬件遥测 <code>WebSocket/REST</code> 协议和降级行为
<code>docker-compose.yml</code>	<code>Redis</code> 、 <code>server</code> 、 <code>worker</code> 和模型目录挂载部署方式

表 7 报告初稿的主要代码依据

12 附录 B：后续补充材料清单

最终提交版建议继续补充以下材料：

- 视觉模型训练集说明、标注规范、训练参数、验证集指标和典型失败案例。
- 至少 6 类电路 demo 的端到端测试表，包括正确接线、缺元件、错孔位、短路、极性反接和未完成电路。
- DK-2500 实测性能曲线，包括端到端延迟、CPU/内存/iGPU/NPU 占用、功耗和热稳定性。
- 前端页面截图，包括学生上传、孔位修正、拓扑可视化、教师 dashboard、Agent 问答和遥测大屏。
- 与传统人工巡查、单纯图像分类或单纯 RAG 问答方案的对比分析。
- 论文规范所需的致谢、外文翻译、开题报告和查重前格式校对材料。